

UNITED STATES DISTRICT COURT
FOR THE DISTRICT OF COLUMBIA

JOSEPH KIRCHNER,

Plaintiff,

v.

MIKE JOHNSON, in his individual and official
capacity as Speaker of the United States House of
Representatives;

DONALD J. TRUMP, in his individual capacity as
former and current President of the United States;

PAM BONDI, in her individual and official
capacity as Attorney General of the United States;

BRENDAN CARR, in his individual and official
capacity as Chairman of the Federal
Communications Commission;

THE UNITED STATES HOUSE OF
REPRESENTATIVES;

ANTHROPIC PBC;

OPENAI, INC.;

APPLE INC.;

COMCAST CABLE COMMUNICATIONS, LLC;

METR (MODEL EVALUATION & THREAT
RESEARCH);

and DOES 1-20, unknown co-conspirators,

Defendants.

Case No. 1:25-cv-02735-ACR

**EXHIBIT METR-13: AI
SAFETY TRANSPARENCY
RESEARCH (3 OF 3)**

Inverse Reward Design

Dylan Hadfield-Menell Smitha Milli Pieter Abbeel* Stuart Russell Anca D. Dragan

Department of Electrical Engineering and Computer Science
University of California, Berkeley
Berkeley, CA 94709

{dhm, smilli, pabbeel, russell, anca}@cs.berkeley.edu

Abstract

Autonomous agents optimize the reward function we give them. What they don't know is how hard it is for us to design a reward function that actually captures what we want. When designing the reward, we might think of some specific training scenarios, and make sure that the reward will lead to the right behavior in *those* scenarios. Inevitably, agents encounter *new* scenarios (e.g., new types of terrain) where optimizing that same reward may lead to undesired behavior. Our insight is that reward functions are merely *observations* about what the designer *actually* wants, and that they should be interpreted in the context in which they were designed. We introduce *inverse reward design* (IRD) as the problem of inferring the true objective based on the designed reward and the training MDP. We introduce approximate methods for solving IRD problems, and use their solution to plan risk-averse behavior in test MDPs. Empirical results suggest that this approach can help alleviate negative side effects of misspecified reward functions and mitigate reward hacking.

1 Introduction

Robots² are becoming more capable of optimizing their reward functions. But along with that comes the burden of making sure we specify these reward functions correctly. Unfortunately, this is a notoriously difficult task. Consider the example from Figure 1. Alice, an AI engineer, wants to build a robot, we'll call it Rob, for mobile navigation. She wants it to reliably navigate to a target location and expects it to primarily encounter grass lawns and dirt pathways. She trains a perception system to identify each of these terrain types and then uses this to define a reward function that incentivizes moving towards the target quickly, avoiding grass where possible. When Rob is deployed into the world, it encounters a novel terrain type; for dramatic effect, we'll suppose that it is lava. The terrain prediction goes haywire on this out-of-distribution input and generates a meaningless classification which, in turn, produces an arbitrary reward evaluation. As a result, Rob might then drive to its demise. This failure occurs because the reward function Alice *specified* implicitly through the terrain predictors, which ends up outputting arbitrary values for lava, is *different* from the one Alice *intended*, which would actually penalize traversing lava.

In the terminology from Amodei et al. (2016), this is a *negative side effect* of a *misspecified reward* — a failure mode of reward design where leaving out important aspects leads to poor behavior. Examples date back to King Midas, who wished that everything he touched turn to gold, leaving out that he didn't mean his food or family. Another failure mode is *reward hacking*, which happens when, e.g., a vacuum cleaner ejects collected dust so that it can collect even more (Russell & Norvig, 2010), or a racing boat in a game loops in place to collect points instead of actually winning the race (Amodei & Clark, 2016). Short of requiring that the reward designer anticipate and penalize all possible misbehavior in advance, how can we alleviate the impact of such reward misspecification?

*OpenAI, International Computer Science Institute (ICSI)

²Throughout this paper, we will use robot to refer generically to any artificial agent.

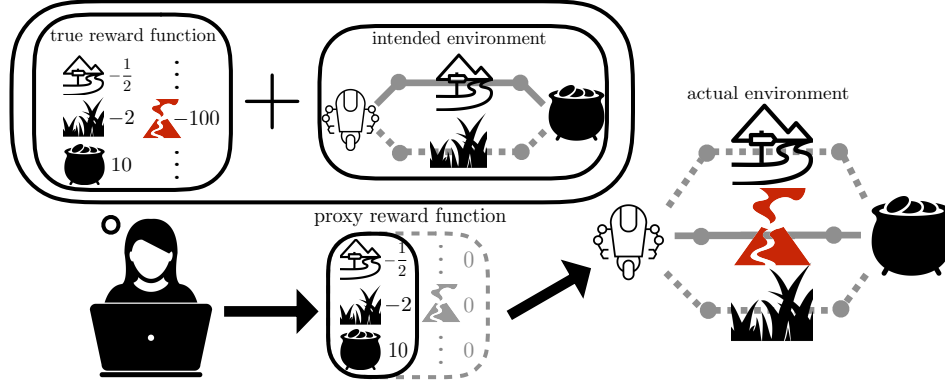


Figure 1: An illustration of a *negative side effect*. Alice designs a reward function so that her robot navigates to the pot of gold and prefers dirt paths. She does not consider that her robot might encounter lava in the real world and leaves that out of her reward specification. The robot maximizing this *proxy* reward function drives through the lava to its demise. In this work, we formalize the (Bayesian) *inverse reward design* (IRD) problem as the problem of inferring (a distribution on) the *true* reward function from the proxy. We show that IRD can help mitigate unintended consequences from misspecified reward functions like negative side effects and reward hacking.

We leverage a key insight: that the designed reward function should merely be an observation about the intended reward, rather than the definition; and should be interpreted in the context in which it was designed. First, a robot should have uncertainty about its reward function, instead of treating it as fixed. This enables it to, e.g., be risk-averse when planning in scenarios where it is not clear what the right answer is, or to ask for help. Being uncertain about the true reward, however, is only half the battle. To be effective, a robot must acquire the *right* kind of uncertainty, i.e. know what it knows and what it doesn't. We propose that the 'correct' shape of this uncertainty depends on the environment for which the reward was designed.

In Alice's case, the situations where she tested Rob's learning behavior did not contain lava. Thus, the lava-avoiding reward would have produced the *same behavior* as Alice's designed reward function in the (lava-free) environments that Alice considered. A robot that knows the settings it was evaluated in should also know that, even though the designer specified a lava-agnostic reward, they might have actually meant the lava-avoiding reward. Two reward functions that would produce similar behavior in the training environment should be treated as equally likely, regardless of which one the designer actually specified. We formalize this in a probabilistic model that relates the proxy (designed) reward to the true reward via the following assumption:

Assumption 1. *Proxy reward functions are likely to the extent that they lead to high **true** utility behavior in the **training** environment.*

Formally, we assume that the observed proxy reward function is the approximate solution to a *reward design problem* (Singh et al., 2010). Extracting the true reward is the *inverse* reward design problem.

The idea of using human behavior as observations about the reward function is far from new. Inverse reinforcement learning uses human demonstrations (Ng & Russell, 2000; Ziebart et al., 2008), shared autonomy uses human operator control signals (Javdani et al., 2015), preference-based reward learning uses answers to comparison queries (Jain et al., 2015), and even what the human wants (Hadfield-Menell et al., 2017). We observe that, *even when the human behavior is to actually write down a reward function*, this should still be treated as an observation, demanding its own observation model.

Our paper makes three contributions. First, we define the *inverse reward design* (IRD) problem as the problem of inferring the true reward function given a proxy reward function, an intended decision problem (e.g., an MDP), and a set of possible reward functions. Second, we propose a solution to IRD and justify how an intuitive algorithm which treats the proxy reward as a set of expert demonstrations can serve as an effective approximation. Third, we show that this inference approach, combined with risk-averse planning, leads to algorithms that are robust to misspecified rewards, alleviating both negative side effects as well as reward hacking. We build a system that 'knows-what-it-knows' about reward evaluations that automatically detects and avoids distributional shift in situations with high-dimensional features. Our approach substantially outperforms the baseline of literal reward interpretation.

2 Inverse Reward Design

Definition 1. (Markov Decision Process Puterman (2009)) A (finite-horizon) Markov decision process (MDP), M , is a tuple $M = \langle \mathcal{S}, \mathcal{A}, T, r, H \rangle$. $\mathcal{S}$ is a set of states. $\mathcal{A}$ is a set of actions. T is a probability distribution over the next state, given the previous state and action. We write this as $T(s_{t+1}|s_t, a)$. r is a reward function that maps states to rewards $r : \mathcal{S} \mapsto \mathbb{R}$. $H \in \mathbb{Z}_+$ is the finite planning horizon for the agent.

A solution to M is a *policy*: a mapping from the current timestep and state to a distribution over actions. The optimal policy maximizes the expected sum of rewards. We will use ξ to represent trajectories. In this work, we consider reward functions that are linear combinations of feature vectors $\phi(\xi)$. Thus, the reward for a trajectory, given weights w , is $r(\xi; w) = w^\top \phi(\xi)$.

The MDP formalism defines optimal behavior, given a reward function. However, it provides no information about where this reward function comes from (Singh et al., 2010). We refer to an MDP without rewards as a *world model*. In practice, a system designer needs to select a reward function that encapsulates the intended behavior. This process is *reward engineering* or *reward design*.

Definition 2. (Reward Design Problem (Singh et al., 2010)) A reward design problem (RDP) is defined as a tuple $P = \langle r^*, \tilde{M}, \tilde{\mathcal{R}}, \pi(\cdot|\tilde{r}, \tilde{M}) \rangle$. r^* is the true reward function. $\tilde{M}$ is a world model. $\tilde{\mathcal{R}}$ is a set of proxy reward functions. $\pi(\cdot|\tilde{r}, \tilde{M})$ is an agent model, that defines a distribution on trajectories given a (proxy) reward function and a world model.

In an RDP, the designer believes that an agent, represented by the policy $\pi(\cdot|\tilde{r}, \tilde{M})$, will be deployed in $\tilde{M}$. She must specify a *proxy reward function* $\tilde{r} \in \tilde{\mathcal{R}}$ for the agent. Her goal is to specify $\tilde{r}$ so that $\pi(\cdot|\tilde{r}, \tilde{M})$ obtains high reward according to the true reward function r^* . We let $\tilde{w}$ represent weights for the proxy reward function and w^* represent weights for the true reward function.

In this work, our motivation is that system designers are fallible, so we should not expect that they perfectly solve the reward design problem. Instead we consider the case where the system designer is approximately optimal at solving a known RDP, which is distinct from the MDP that the robot currently finds itself in. By inverting the reward design process to infer (a distribution on) the true reward function r^* , the robot can understand where its reward evaluations have high variance and plan to avoid those states. We refer to this inference problem as the *inverse reward design problem*:

Definition 3. (Inverse Reward Design) The inverse reward design (IRD) problem is defined by a tuple $\langle \tilde{\mathcal{R}}, \tilde{M}, \tilde{\mathcal{R}}, \pi(\cdot|\tilde{r}, \tilde{M}), \tilde{r} \rangle$. $\tilde{\mathcal{R}}$ is a space of possible reward functions. $\tilde{M}$ is a world model. $\langle -, \tilde{M}, \tilde{\mathcal{R}}, \pi(\cdot|\tilde{r}, \tilde{M}) \rangle$ partially specifies an RDP P , with an unobserved reward function $r^* \in \mathcal{R}$. $\tilde{r} \in \tilde{\mathcal{R}}$ is the observed proxy reward that is an (approximate) solution to P .

In solving an IRD problem, the goal is to recover r^* . We will explore Bayesian approaches to IRD, so we will assume a prior distribution on r^* and infer a posterior distribution on r^* given $\tilde{r} \sim P(r^*|\tilde{r}, \tilde{M})$.

3 Related Work

Optimal reward design. Singh et al. (2010) formalize and study the problem of designing optimal rewards. They consider a designer faced with a distribution of environments, a class of reward functions to give to an agent, and a *fitness* function. They observe that, in the case of bounded agents, it may be optimal to select a proxy reward that is distinct from the fitness function. Sorg et al. (2010) and subsequent work has studied the computational problem of selecting an optimal proxy reward.

In our work, we consider an alternative situation where the *system designer* is the bounded agent. In this case, the proxy reward function is distinct from the fitness function – the true utility function in our terminology – because system designers can make mistakes. IRD formalizes the problem of determining a true utility function given an observed proxy reward function. This enables us to design agents that are robust to misspecifications in their reward function.

Inverse reinforcement learning. In *inverse reinforcement learning* (IRL) (Ng & Russell, 2000; Ziebart et al., 2008; Evans et al., 2016; Syed & Schapire, 2007) the agent observes demonstrations of (approximately) optimal behavior and infers the reward function being optimized. IRD is a similar

problem, as both approaches infer an unobserved reward function. The difference is in the observation: IRL observes behavior, while IRD directly observes a reward function. Key to IRD is assuming that this observed reward incentivizes behavior that is approximately optimal with respect to the true reward. In Section 4.2, we show how ideas from IRL can be used to approximate IRD. Ultimately, we consider both IRD and IRL to be complementary strategies for *value alignment* (Hadfield-Menell et al., 2016): approaches that allow designers or users to communicate preferences or goals.

Pragmatics. The *pragmatic* interpretation of language is the interpretation of a phrase or utterance in the context of alternatives (Grice, 1975). For example, the utterance “some of the apples are red” is often interpreted to mean that “not all of the apples are red” although this is not literally implied. This is because, in context, we typically assume that a speaker who meant to say “all the apples are red” would simply say so.

Recent models of pragmatic language interpretation use two levels of Bayesian reasoning (Frank et al., 2009; Goodman & Lassiter, 2014). At the lowest level, there is a literal listener that interprets language according to a shared literal definition of words or utterances. Then, a speaker selects words in order to convey a particular meaning to the literal listener. To model pragmatic inference, we consider the probable meaning of a given utterance from this speaker. We can think of IRD as a model of pragmatic reward interpretation: the speaker in pragmatic interpretation of language is directly analogous to the reward designer in IRD.

4 Approximating the Inference over True Rewards

We solve IRD problems by formalizing Assumption 1: the idea that proxy reward functions are likely to the extent that they incentivize high utility behavior in the training MDP. This will give us a probabilistic model for how $\tilde{w}$ is generated from the true w^* and the training MDP $\tilde{M}$. We will invert this probability model to compute a distribution $P(w = w^* | \tilde{w}, \tilde{M})$ on the true utility function.

4.1 Observation Model

Recall that $\pi(\xi | \tilde{w}, \tilde{M})$ is the designer’s model of the probability that the robot will select trajectory ξ , given proxy reward $\tilde{w}$. We will assume that $\pi(\xi | \tilde{w}, \tilde{M})$ is the maximum entropy trajectory distribution from Ziebart et al. (2008), i.e. the designer models the robot as approximately optimal: $\pi(\xi | \tilde{w}, \tilde{M}) \propto \exp(w^\top \phi(\xi))$. An optimal designer chooses $\tilde{w}$ to maximize expected true value, i.e. $\mathbb{E}[w^{*\top} \phi(\xi) | \xi \sim \pi(\xi | \tilde{w}, \tilde{M})]$ is high. We model an approximately optimal designer:

$$P(\tilde{w} | w^*, \tilde{M}) \propto \exp \left(\beta \mathbb{E} \left[w^{*\top} \phi(\xi) | \xi \sim \pi(\xi | \tilde{w}, \tilde{M}) \right] \right) \quad (1)$$

with β controlling how close to optimal we assume the person to be. This is now a formal statement of Assumption 1. w^* can be pulled out of the expectation, so we let $\tilde{\phi} = \mathbb{E}[\phi(\xi) | \xi \sim \pi(\xi | \tilde{w}, \tilde{M})]$. Our goal is to invert (1) and sample from (or otherwise estimate) $P(w^* | \tilde{w}, \tilde{M}) \propto P(\tilde{w} | w^*, \tilde{M}) P(w^*)$. The primary difficulty this entails is that we need to know the *normalized* probability $P(\tilde{w} | w^*, \tilde{M})$. This depends on its normalizing constant, $\tilde{Z}(w)$, which integrates over possible proxy rewards.

$$P(w = w^* | \tilde{w}, \tilde{M}) \propto \frac{\exp \left(\beta w^\top \tilde{\phi} \right)}{\tilde{Z}(w)} P(w), \quad \tilde{Z}(w) = \int_{\tilde{w}} \exp \left(\beta w^\top \tilde{\phi} \right) d\tilde{w}. \quad (2)$$

4.2 Efficient approximations to the IRD posterior

To compute $P(w = w^* | \tilde{w}, \tilde{M})$, we must compute $\tilde{Z}$, which is intractable if $\tilde{w}$ lies in an infinite or large finite set. Notice that computing the value of the integrand for $\tilde{Z}$ is highly non-trivial as it involves solving a planning problem. This is an example of what is referred to as a *doubly-intractable* likelihood (Murray et al., 2006). We consider two methods to approximate this normalizing constant.

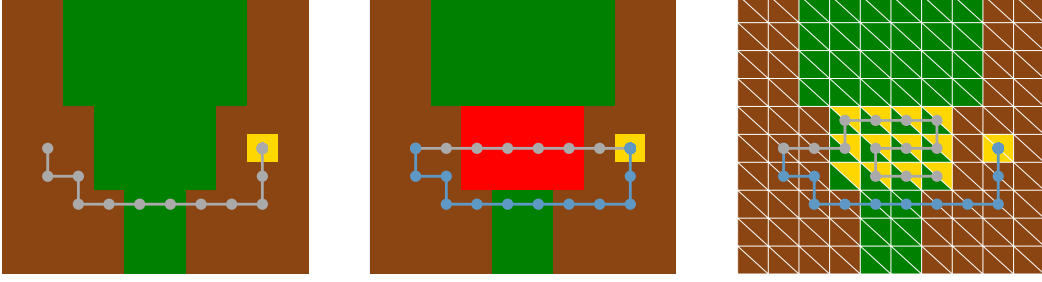


Figure 2: An example from the Lavaland domain. **Left:** The training MDP where the designer specifies a proxy reward function. This incentivizes movement toward targets (yellow) while preferring dirt (brown) to grass (green), and generates the gray trajectory. **Middle:** The testing MDP has lava (red). The proxy does not penalize lava, so optimizing it makes the agent go straight through (gray). This is a negative side effect, which the IRD agent avoids (blue): it treats the proxy as an observation in the context of the training MDP, which makes it realize that it cannot trust the (implicit) weight on lava. **Right:** The testing MDP has cells in which two sensor indicators no longer correlate: they look like grass to one sensor but target to the other. The proxy puts weight on the first, so the literal agent goes to these cells (gray). The IRD agent knows that it can't trust the distinction and goes to the target on which both sensors agree (blue).

Sample to approximate the normalizing constant. This approach, inspired by methods in approximate Bayesian computation (Sunnåker et al., 2013), samples a finite set of weights $\{w_i\}$ to approximate the integral in Equation 2. We found empirically that it helped to include the candidate sample w in the sum. This leads to the normalizing constant

$$\hat{Z}(w) = \exp(\beta w^\top \phi_w) + \sum_{i=0}^{N-1} \exp(\beta w^\top \phi_i). \quad (3)$$

Where ϕ_i and ϕ_w are the vector of feature counts realized optimizing w_i and w respectively.

Bayesian inverse reinforcement learning. During inference, the normalizing constant serves a calibration purpose: it computes how good the behavior produced by all proxy rewards in that MDP would be with respect to the true reward. Reward functions which increase the reward for *all* trajectories are not preferred in the inference. This creates an invariance to linear shifts in the feature encoding. If we were to change the MDP by shifting features by some vector ϕ_0 , $\phi \leftarrow \phi + \phi_0$, the posterior over w would remain the same.

We can achieve a similar calibration and maintain the same property by directly integrating over the possible trajectories in the MDP:

$$Z(w) = \left(\int_{\xi} \exp(w^\top \phi(\xi)) d\xi \right)^\beta; \quad \hat{P}(w|\tilde{w}) \propto \frac{\exp(\beta w^\top \tilde{\phi})}{Z(w)} \quad (4)$$

Proposition 1. *The posterior distribution that the IRD model induces on w^* (i.e., Equation 2) and the posterior distribution induced by IRL (i.e., Equation 4) are invariant to linear translations of the features in the training MDP.*

Proof. First, we observe that this shift does not change the behavior of the planning agent due to linearity of the Bellman backup operation, i.e., $\tilde{\phi}' = \tilde{\phi} + \phi_0$. In Equation 2 linearity of expectation allows us to pull ϕ_0 out of the expectation to compute $\tilde{\phi}$:

$$\frac{\exp(\beta w^\top \tilde{\phi}')}{\int_{\tilde{w}} \exp(\beta w^\top \tilde{\phi}') d\tilde{w}} = \frac{\exp(\beta w^\top \phi_0) \exp(\beta w^\top \tilde{\phi})}{\int_{\tilde{w}} \exp(\beta w^\top \phi_0) \exp(\beta w^\top \tilde{\phi}) d\tilde{w}} \quad (5)$$

$$= \frac{\exp(\beta w^\top \tilde{\phi})}{\int_{\tilde{w}} \exp(\beta w^\top \tilde{\phi}) d\tilde{w}} \quad (6)$$

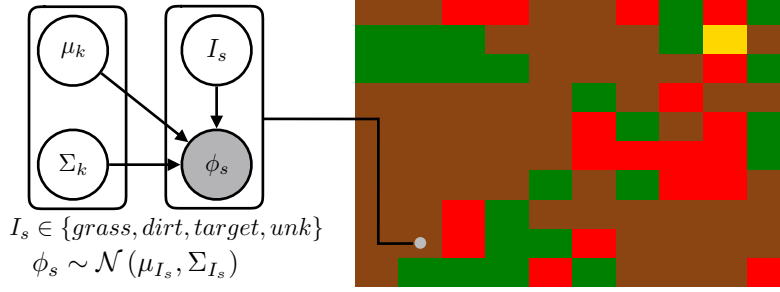


Figure 3: Our challenge domain with latent rewards. Each terrain type (grass, dirt, target, lava) induces a different distribution over high-dimensional features: $\phi_s \sim \mathcal{N}(\mu_{I_s}, \Sigma_{I_s})$. The designer never builds an indicator for lava, and yet the agent still needs to avoid it in the test MDPs.

This shows that Equation 2 is invariant to constant shifts in the feature function. The same argument applies to Equation 4. $\square$

This choice of normalizing constant approximates the posterior to an IRD problem with the posterior from maximum entropy IRL (Ziebart et al., 2008). The result has an intuitive interpretation. The proxy $\tilde{w}$ determines the average feature counts for a hypothetical dataset of expert demonstrations and β determines the effective size of that dataset. The agent solves $\tilde{M}$ with reward $\tilde{w}$ and computes the corresponding feature expectations $\tilde{\phi}$. The agent then pretends like it got β demonstrations with features counts $\tilde{\phi}$, and runs IRL. The more the robot believes the human is good at reward design, the more demonstrations it pretends to have gotten from the person. The fact that reducing the proxy to behavior in $\tilde{M}$ approximates IRD is not surprising: the main point of IRD is that the proxy *reward* is merely a statement about what *behavior* is good in the training environment.

5 Evaluation

5.1 Experimental Testbed

We evaluated our approaches in a model of the scenario from Figure 1 that we call *Lavaland*. Our system designer, Alice, is programming a mobile robot, Rob. We model this as a gridworld with movement in the four cardinal directions and four terrain types: target, grass, dirt, and lava. The true objective for Rob, w^* , encodes that it should get to the target quickly, stay off the grass, and avoid lava. Alice designs a proxy that performs well in a training MDP that does not contain lava. Then, we measure Rob’s performance in a test MDP that *does* contain lava. Our results show that combining IRD and risk-averse planning creates incentives for Rob to avoid unforeseen scenarios.

We experiment with four variations of this environment: two proof-of-concept conditions in which the reward is misspecified, but the agent has direct access to feature indicators for the different categories (i.e. conveniently having a feature for lava); and two challenge conditions, in which the right features are *latent*; the reward designer does not build an indicator for lava, but by reasoning in the raw observation space and then using risk-averse planning, the IRD agent still avoids lava.

5.1.1 Proof-of-Concept Domains

These domains contain feature indicators for the four categories: grass, dirt, target, and lava.

Side effects in Lavaland. Alice expects Rob to encounter 3 types of terrain: grass, dirt, and target, and so she only considers the training MDP from Figure 2 (left). She provides a $\tilde{w}$ to encode a trade-off between path length and time spent on grass.

The training MDP contains no lava, but it is introduced when Rob is deployed. An agent that treats the proxy reward literally might go on the lava in the test MDP. However, an agent that runs IRD will know that it can’t trust the weight on the lava indicator, since all such weights would produce the same behavior in the training MDP (Figure 2, middle).

Reward Hacking in Lavaland. Reward hacking refers generally to reward functions that can be gamed or tricked. To model this within Lavaland, we use features that are correlated in the training domain but are uncorrelated in the testing environment. There are 6 features: three from one sensor and three from another sensor. In the training environment the features from both sensors are correct indicators of the state’s terrain category (grass, dirt, target).

At test time, this correlation gets broken: lava looks like the target category to the second sensor, but the grass category to the first sensor. This is akin to how in a racing game (Amodei & Clark, 2016), winning and game points can be correlated at reward design time, but test environments might contain loopholes for maximizing points without winning. We want agents to hedge their bets between winning and points, or, in Lavaland, between the two sensors. An agent that treats the proxy reward function literally might go to these new cells if they are closer. In contrast, an agent that runs IRD will know that a reward function with the same weights put on the first sensor is just as likely as the proxy. Risk averse planning makes it go to the target for which both sensors agree (Figure 2, right).

5.1.2 Challenge Domain: Latent Rewards, No More Feature Indicators

The previous examples allow us to explore reward hacking and negative side effects in an isolated experiment, but are unrealistic as they assume the existence of a feature indicator for unknown, unplanned-for terrain. To investigate misspecified objectives in a more realistic setting, we shift to the terrain type being latent, and inducing raw observations: we use a model where the terrain category determines the mean and variance of a multivariate Gaussian distribution over observed features. Figure 3 shows a depiction of this scenario. The designer has in mind a proxy reward on dirt, target, and grass, but *forgets that lava might exist*. We consider two realistic ways through which a designer might actually specify the proxy reward function, which is based on the terrain types that the robot does not have access to: 1) directly on the **raw observations** — collect samples of the training terrain types (dirt, grass, target) and train a (linear) reward predictor; or 2) **classifier features** — build a classifier to classify terrain as dirt, grass, or target, and define a proxy on its output.

Note that this domain allows for both negative side effects and reward hacking. Negative side effects can occur because the feature distribution for lava is different from the feature distribution for the three safe categories, and the proxy reward is trained only on the three safe categories. Thus in the testing MDP, the evaluation of the lava cells will be arbitrary so maximizing the proxy reward will likely lead the agent into lava. Reward hacking occurs when features that are correlated for the safe categories are uncorrelated for the lava category.

5.2 Experiment

Lavaland Parameters. We defined a distribution on map layouts with a log likelihood function that prefers maps where neighboring grid cells are the same. We mixed this log likelihood with a quadratic cost for deviating from a target ratio of grid cells to ensure similar levels of the lava feature in the testing MDPs. Our training MDP is 70% dirt and 30% grass. Our testing MDP is 5% lava, 66.5% dirt, and 28.5% grass.

In the proof-of-concept experiments, we selected the proxy reward function uniformly at random. For latent rewards, we picked a proxy reward function that evaluated to +1 for target, +.1 for dirt, and −.2 for grass. To define a proxy on raw observations, we sampled 1000 examples of grass, dirt, and target and did a linear regression. With classifier features, we simply used the target rewards as the weights on the classified features. We used 50 dimensions for our feature vectors. We selected trajectories via *risk-averse trajectory optimization*. Details of our planning method, and our approach and rationale in selecting it can be found in the supplementary material.

IVs and DVs. We measured the fraction of runs that encountered a lava cell on the test MDP as our dependent measure. This tells us the proportion of trajectories where the robot gets ‘tricked’ by the misspecified reward function; if a grid cell has never been seen then a conservative robot should plan to avoid it. We manipulate two factors: **literal-optimizer** and **Z-approx**. **literal-optimizer** is true if the robot interprets the proxy reward literally and false otherwise. **Z-approx** varies the approximation technique used to compute the IRD posterior. It varies across the two levels described in Section 4.2: sample to approximate the normalizing constant (**Sample-Z**) or use the normalizing constant from maximum entropy IRL (**MaxEnt-Z**) (Ziebart et al., 2008).

Results. Figure 4 compares the approaches. On the left, we see that IRD alleviates negative side effects (avoids the lava) and reward hacking (does not go as much on cells that look deceptively like

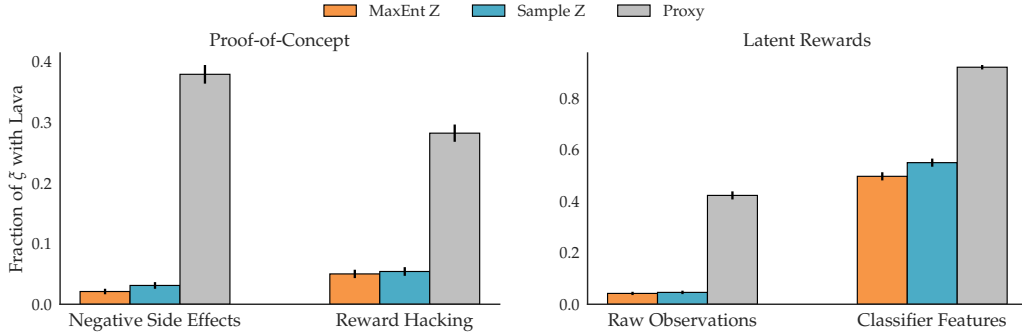


Figure 4: The results of our experiment comparing our proposed method to a baseline that directly plans with the proxy reward function. By solving an inverse reward design problem, we are able to create generic incentives to avoid unseen or novel states.

the target to one of the sensors). This is important, in that the same inference method generalizes across different consequences of misspecified rewards. Figure 2 shows example behaviors.

In the more realistic latent reward setting, the IRD agent avoids the lava cells despite the designer forgetting to penalize it, and despite not even having an indicator for it: because lava is latent in the space, and so reward functions that would *implicitly* penalize lava are as likely as the one actually specified, risk-averse planning avoids it.

We also see a distinction between raw observations and classifier features. The first essentially matches the proof-of-concept results (note the different axes scales), while the latter is much more difficult across all methods. The proxy performs worse because each grid cell is classified before being evaluated, so there is a relatively good chance that at least one of the lava cells is misclassified as target. IRD performs worse because the behaviors considered in inference plan in the already classified terrain: a non-linear transformation of the features. The inference must both determine a good linear reward function to match the behavior *and* discover the corresponding uncertainty about it. When the proxy is a linear function of raw observations, the first job is considerably easier.

6 Discussion

Summary. In this work, we motivated and introduced the *Inverse Reward Design* problem as an approach to mitigate the risk from misspecified objectives. We introduced an observation model, identified the challenging inference problem this entails, and gave several simple approximation schemes. Finally, we showed how to use the solution to an inverse reward design problem to avoid side effects and reward hacking in a 2D navigation problem. We showed that we are able to avoid these issues reliably in simple problems where features are binary indicators of terrain type. Although this result is encouraging, in real problems we won’t have convenient access to binary indicators for what matters. Thus, our challenge evaluation domain gave the robot access to only a high-dimensional observation space. The reward designer specified a reward based on this observation space which forgets to penalize a rare but catastrophic terrain. IRD inference still enabled the robot to understand that rewards which would implicitly penalize the catastrophic terrain are also likely.

Limitations and future work. IRD gives the robot a posterior distribution over reward functions, but much work remains in understanding how to best leverage this posterior. Risk-averse planning can work sometimes, but it has the limitation that the robot does not just avoid bad things like lava, it also avoids potentially good things, like a giant pot of gold. We anticipate that leveraging the IRD posterior for follow-up queries to the reward designer will be key to addressing misspecified objectives.

Another limitation stems from the complexity of the environments and reward functions considered here. The approaches we used in this work rely on explicitly solving a planning problem, and this is a bottleneck during inference. In future work, we plan to explore the use of different agent models that plan approximately or leverage, e.g., meta-learning (Duan et al., 2016) to scale IRD up to complex environments. Another key limitation is the use of linear reward functions. We cannot expect IRD to perform well unless the prior places weights on (a reasonable approximation to) the true reward function. If, e.g., we encoded terrain types as RGB values in Lavaland, there is unlikely to be a reward function in our hypothesis space that represents the true reward well.

Finally, this work considers one relatively simple error model for the designer. This encodes some implicit assumptions about the nature and likelihood of errors (e.g., IID errors). In future work, we plan to investigate more sophisticated error models that allow for systematic biased errors from the designer and perform human subject studies to empirically evaluate these models.

Overall, we are excited about the implications IRD has not only in the short term, but also about its contribution to the general study of the value alignment problem.

Acknowledgements

This work was supported by the Center for Human Compatible AI and the Open Philanthropy Project, the Future of Life Institute, AFOSR, and NSF Graduate Research Fellowship Grant No. DGE 1106400.

References

- Amodei, Dario and Clark, Jack. Faulty Reward Functions in the Wild. <https://blog.openai.com/faulty-reward-functions/>, 2016.
- Amodei, Dario, Olah, Chris, Steinhardt, Jacob, Christiano, Paul, Schulman, John, and Mané, Dan. Concrete Problems in AI Safety. *CoRR*, abs/1606.06565, 2016. URL <http://arxiv.org/abs/1606.06565>.
- Duan, Yan, Schulman, John, Chen, Xi, Bartlett, Peter L., Sutskever, Ilya, and Abbeel, Pieter. RL²: Fast Reinforcement Learning via Slow Reinforcement Learning. *CoRR*, abs/1611.02779, 2016. URL <http://arxiv.org/abs/1611.02779>.
- Evans, Owain, Stuhlmüller, Andreas, and Goodman, Noah D. Learning the Preferences of Ignorant, Inconsistent Agents. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pp. 323–329. AAAI Press, 2016.
- Frank, Michael C, Goodman, Noah D, Lai, Peter, and Tenenbaum, Joshua B. Informative Communication in Word Production and Word Learning. In *Proceedings of the 31st Annual Conference of the Cognitive Science Society*, pp. 1228–1233. Cognitive Science Society Austin, TX, 2009.
- Goodman, Noah D and Lassiter, Daniel. Probabilistic Semantics and Pragmatics: Uncertainty in Language and Thought. *Handbook of Contemporary Semantic Theory*. Wiley-Blackwell, 2, 2014.
- Grice, H. Paul. *Logic and Conversation*, pp. 43–58. Academic Press, 1975.
- Hadfield-Menell, Dylan, Dragan, Anca, Abbeel, Pieter, and Russell, Stuart. Cooperative Inverse Reinforcement Learning. In *Proceedings of the Thirtieth Annual Conference on Neural Information Processing Systems*, 2016.
- Hadfield-Menell, Dylan, Dragan, Anca D., Abbeel, Pieter, and Russell, Stuart J. The Off-Switch Game. In *Proceedings of the International Joint Conference on Artificial Intelligence*, 2017.
- Jain, Ashesh, Sharma, Shikhar, Joachims, Thorsten, and Saxena, Ashutosh. Learning Preferences for Manipulation Tasks from Online Coactive Feedback. *The International Journal of Robotics Research*, 34(10):1296–1313, 2015.
- Javdani, Shervin, Bagnell, J. Andrew, and Srinivasa, Siddhartha S. Shared Autonomy via Hindsight Optimization. In *Proceedings of Robotics: Science and Systems XI*, 2015. URL <http://arxiv.org/abs/1503.07619>.
- Markowitz, Harry M. *Portfolio selection: efficient diversification of investments*, volume 16. Yale university press, 1968.
- Murray, Iain, Ghahramani, Zoubin, and MacKay, David. MCMC for Doubly-Intractable Distributions. In *Proceedings of the Twenty-Second Conference on Uncertainty in Artificial Intelligence*, 2006.
- Ng, Andrew Y and Russell, Stuart J. Algorithms for Inverse Reinforcement Learning. In *Proceedings of the Seventeenth International Conference on Machine Learning*, pp. 663–670, 2000.

- Puterman, Martin L. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 2009.
- Rockafellar, R Tyrrell and Uryasev, Stanislav. Optimization of conditional value-at-risk. *Journal of risk*, 2:21–42, 2000.
- Russell, Stuart and Norvig, Peter. *Artificial Intelligence: A Modern Approach*. Pearson, 2010.
- Singh, Satinder, Lewis, Richard L., , and Barto, Andrew G. Where do rewards come from? In *Proceedings of the International Symposium on AI Inspired Biology - A Symposium at the AISB 2010 Convention*, pp. 111–116, 2010. ISBN 1902956923.
- Sorg, Jonathan, Lewis, Richard L, and Singh, Satinder P. Reward Design via Online Gradient Ascent. In *Proceedings of the Twenty-Third Conference on Neural Information Processing Systems*, pp. 2190–2198, 2010.
- Sunnåker, Mikael, Busetto, Alberto Giovanni, Numminen, Elina, Corander, Jukka, Foll, Matthieu, and Dessimoz, Christophe. Approximate Bayesian Computation. *PLoS Comput Biol*, 9(1):e1002803, 2013.
- Syed, Umar and Schapire, Robert E. A Game-Theoretic Approach to Apprenticeship Learning. In *Proceedings of the Twentieth Conference on Neural Information Processing Systems*, pp. 1449–1456, 2007.
- Syed, Umar, Bowling, Michael, and Schapire, Robert E. Apprenticeship learning using linear programming. In *Proceedings of the 25th International Conference on Machine Learning*, pp. 1032–1039. ACM, 2008.
- Tamar, Aviv, Chow, Yinlam, Ghavamzadeh, Mohammad, and Mannor, Shie. Policy gradient for coherent risk measures. In *Advances in Neural Information Processing Systems*, pp. 1468–1476, 2015.
- Ziebart, Brian D, Maas, Andrew L, Bagnell, J Andrew, and Dey, Anind K. Maximum Entropy Inverse Reinforcement Learning. In *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*, pp. 1433–1438, 2008.

Appendix: Risk Averse Trajectory Optimization

Our overall strategy is to implement a system that ‘knows-what-it-knows’ about reward evaluations. So far we have considered the problem of computing the robot’s uncertainty about reward evaluations. We have not considered the problem of using that reward uncertainty. Here we describe several approaches and highlight an important nuance of planning under a distribution over utility evaluations.

The most straightforward option is to maximize expected reward under this distribution. However, this will ignore the reward uncertainty we have worked so hard to achieve: planning in expectation under a distribution over utility is equivalent to planning with the mean of that distribution. Instead we will plan in a risk averse fashion that penalizes trajectories which have high variance over their utility. Of course, risk averse planning is a rich field with a variety of approaches (Markowitz, 1968; Rockafellar & Uryasev, 2000; Tamar et al., 2015). In future work, we intended to explore a variety of risk averse planning methods and evaluate their relative pros and cons for our application.

In this work, we will take a simple approach: given a set of weights $\{w_i\}$ sampled from our posterior $P(w|\tilde{w}, \tilde{M})$, we will have the agent compute a trajectory that maximizes reward under the *worst case* w_i in our set. We can do this in one of two ways.

Trajectory-wide reward. Given a set of weights $\{w_i\}$ sampled from our posterior $P(w|\tilde{w})$, we will have the agent compute a trajectory that maximizes reward under the *worst case* w_i in our set:

$$\xi^* = \operatorname{argmax}_{\xi} \min_{w \in \{w_i\}} w^\top \phi(\xi). \quad (7)$$

This planning problem is no longer isomorphic to an MDP, as the reward may not decompose per state. Trajectory optimization in this case can be done via the linear programming approach described in Syed et al. (2008).

Time-step independent reward. An alternative is to take minimum over weights on a per state basis:

$$\xi^* = \operatorname{argmax}_{\xi} \sum_{s_t \in \xi} \min_{w \in \{w_i\}} w^\top \phi(s_t). \quad (8)$$

This is more conservative, because it allows the minimizer to pick a different reward for each time step.

Directly applying this approach, however, may lead to poor results. The reason is that, unlike maximizing expected reward, this planning approach will be sensitive to the particular feature encoding used. In maximizing expected reward, shifting all feature by a constant vector ϕ_0 will not change the optimal trajectory. *The same is no longer true for a risk averse approach.*

For example, consider a choice between actions a_1 and a_2 , with features ϕ_1 and ϕ_2 respectively. If we shift the features by a constant value $-\phi_2$ (i.e., set the feature values for the second action to 0), then, unless a_1 is preferred to a_2 for *every* weight in the posterior, the agent will always select the second action. The zero values of feature encodings are typically arbitrary, so this is clearly undesirable behavior.

Intuitively, this is because rewards are not absolute, they are relative. Rewards need a reference point. We thus need to compare the reward $w^\top \phi(\xi)$ to *something*: to the reward of some reference features c_i . We will study three approaches: comparing reward to the initial state, to the training feature counts, and to the expected reward across any trajectory.

Comparing to initial state. One straightforward approach is to take a particular state or trajectory and enforce that it has the same evaluation across each w_i . For example, we can enforce that the features for the initial state state is the 0 vector. This has the desirable property that the agent will remain in place (or try to) when there is very high variance in the reward estimates (i.e., the solution to IRD gives little information about the current optimal trajectory).

Comparing to training feature counts. An third option is to use the expected features $\tilde{\phi}$ as the feature offset. In the case, the agent will default to trying to match the features that it would have observed maximizing $\tilde{w}$ in $\tilde{M}$.

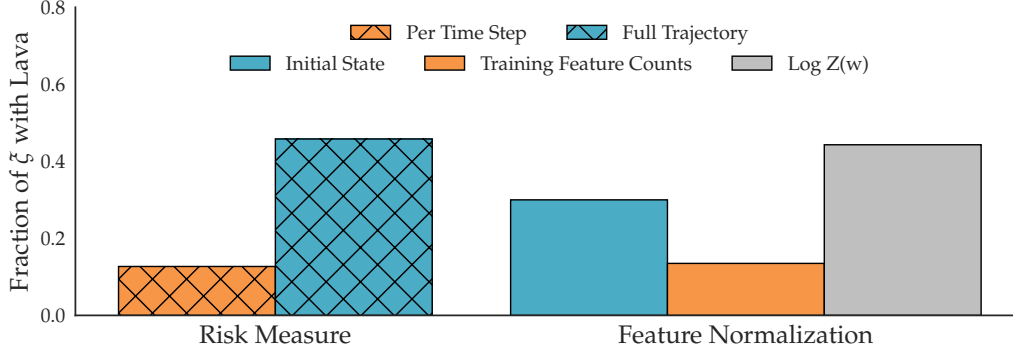


Figure 5: **Left:** We avoid side effects and reward hacking by computing a posterior distribution over reward function and then find a trajectory that performs well under the worst case reward function. This illustrates the impact of selecting this worst case independently per time step or once for the entire trajectory. Taking the minimum per time step increases robustness to the approximate inference algorithms used because we only need one particle in our sample posterior to capture the worst case for each grid cell type. For the full trajectory, we need a single particle to have inferred a worst case for *every* grid cell type at once. **Right:** The impact of changing the offsets c_i . “Initial State” fixes the value of the start state to be 0. “Training Feature Counts” sets an average feature value from the training MDP to be 0. “Log Z(w)” offsets each evaluation by the normalizing from the maximum entropy trajectory distribution. This means that the sum of rewards across a trajectory is the log probability of a trajectory.

Comparing to other trajectories. An alternative is to define c_i as the log of the normalizing constant for the maximum entropy trajectory distribution:

$$c_i = \log \int_{\xi} \exp(w_i^{\top} \phi(\xi)) d\xi. \quad (9)$$

With this choice of c_i , we have that $w_i^{\top} \phi(\xi) - c_i = \log P(\xi|w_i)$. Thus, this approach will select trajectories that compare relatively well to the options under all w_i . Loosely speaking, we can think of it as controlling for the total amount of reward available in the MDP.

Evaluation. Before running the full experiment, we did an initial internal comparison to find the best-performing planning method. We did a full factorial across the factors with the side effect feature encoding and the reward hacking feature encoding.

We found that the biggest overall change came from the **min-granularity** feature. A bar plot is shown in Figure 5 (Left). Independently minimizing per time step was substantially more robust. We hypothesize that this is a downstream effect of the approximate inference used. We sample from our belief to obtain a particle representation of the posterior. Independently minimizing means that we need a single particle to capture the worst case for each grid cell type. Performing this minimization across the full trajectory means that a single particle has to faithfully represent the worst case for *every* grid cell type.

We also saw substantial differences with respect to the **reward-baseline** factor. Figure 5 (Right) shows a bar plot of the results. In this case, setting the common comparison point to be the average feature counts from the training MDP performed best. We believe this is because of the similarity between the train and test scenarios: although there is a new grid cell present, it is still usually possible to find a trajectory that is similar to those available in the training MDP. We hypothesize that correctly making this decision will depend on the situation and we looking forward to exploring this in future work.